PRÁCTICA 3 · INDIVIDUAL · 8 HORAS

Práctica 3 · Lord of EDA

Tablas de dispersión: operaciones de una tabla, operaciones entre dos tablas, otras funciones hash y otras formas de resolver las colisiones

Tema	6. Tablas de dispersión
Modalidad	Individual
Dedicación	8 horas
Web	**https://lordtables.vercel.app**
Entrega	Una única entrega en la PDU, hasta el domingo 13 de diciembre de 2026 a las 23:59
Lenguaje	C, sobre el proyecto Listas_Dispersas de clase

Qué hay que hacer

Sobre las dos tablas de clase, `TablaDispersaLista` (101 cubos con encadenamiento) y `TablaDispersaArray` (20 casillas con exploración lineal y marcas VACIA, OCUPADA y BORRADA), hay que programar las quince funciones que se describen abajo. Las cabeceras ya están en los `.h` del proyecto base y su sitio en los `.c` está marcado con `TODO`.

Cada ejercicio se prueba con una misión de la web: un programa ya hecho lee los eventos de la misión, llama a tus funciones y escribe un registro. La web anima ese registro evento a evento y dice qué está bien y qué no. Las misiones 5, 9, 12 y 16 juntan varios ejercicios.

Cómo se trabaja

1. En https://lordtables.vercel.app, despliega «Las órdenes de Merla» y descarga el proyecto base. Ábrelo en CLion con File > Open (la carpeta entera). Es el mismo proyecto para toda la práctica.
2. Programa cada ejercicio en `TablaDispersaLista.c` y `TablaDispersaArray.c`.
3. Elige en CLion el target de su misión (`mision1` a `mision16`) y ejecútalo. Deja su registro en `registros/`.
4. En la web, entra en esa misión y sube el registro. La nota orientativa sale al final de la animación. Corrige y repite.

Entrega

- Una única entrega en la tarea de la práctica en la PDU, como muy tarde el **domingo 13 de diciembre de 2026 a las 23:59**.
- Un archivo ZIP con **la carpeta del proyecto entera**: el código y la carpeta `registros/`. Puedes quitar la carpeta `cmake-build-debug`.
- Nombre del archivo: **ListasDispersas_numeroAlumno.zip**, con tu número de alumno. Por ejemplo, `ListasDispersas_123456.zip`.
- Puedes subirlo al acabar cada sesión y volver a subirlo: cuenta el último que haya al cerrar el plazo.
- Los registros tienen que salir de ejecutar el código que entregas.

Ejercicios

Ejercicio 1 · El elemento más grande se prueba con `mision1`

> TABLADISPERSALISTA.C
>
> ```
> int masAbundanteHashLista(HashLista *hashLista, int *clave, int *elemento);
> ```
>
> Recorre todos los cubos y deja en *clave y *elemento el par con el elemento más grande; si hay empate, el de la clave más pequeña. Devuelve 1, o 0 si la tabla está vacía (y entonces no toca *clave ni *elemento).

> TABLADISPERSAARRAY.C
>
> ```
> int masAbundanteHashArray(HashArray *hashArray, int *clave, int *elemento);
> ```
>
> Lo mismo, mirando solo las casillas OCUPADA. Devuelve 1, o 0 si no hay ninguna.

Ejercicio 2 · Medir las colisiones se prueba con `mision2`

> TABLADISPERSALISTA.C
>
> ```
> int cuboMasLargoHashLista(HashLista *hashLista, int *vacios);
> ```
>
> Devuelve cuántos nodos tiene la lista más larga y deja en *vacios cuántos cubos están a NULL.

> TABLADISPERSAARRAY.C
>
> ```
> int grupoMasLargoHashArray(HashArray *hashArray, int *borradas);
> ```
>
> Devuelve la longitud del grupo más largo de casillas seguidas que no están VACIA (OCUPADA o BORRADA), mirando de la casilla 0 a la última sin dar la vuelta, y deja en *borradas cuántas BORRADA hay.

Ejercicio 3 · Cambiar la clave se prueba con `mision3`

TABLADISPERSALISTA.C

```
int cambiarClaveHashLista(HashLista *hashLista, int vieja, int nueva);
```

Pasa lo de la clave vieja a la nueva. Si la nueva no está, desengancha el MISMO nodo, le cambia la clave y lo engancha al principio del cubo de la nueva. Si la nueva ya está, le suma el elemento y libera el nodo viejo. Devuelve 1, o 0 si la vieja no está; si vieja == nueva, no hace nada y devuelve 1.

TABLADISPERSAARRAY.C

```
int cambiarClaveHashArray(HashArray *hashArray, int vieja, int nueva);
```

La casilla de la vieja pasa a BORRADA; la nueva suma el elemento si ya estaba o se inserta como en insertarHashArray. Devuelve lo mismo que la de listas.

Ejercicio 4 · Quitar los que tienen poco se prueba con `mision4`

TABLADISPERSALISTA.C

```
int eliminarSiMenorHashLista(HashLista *hashLista, int minimo);
```

Quita y libera todos los nodos cuyo elemento sea menor que minimo. Devuelve cuántos ha quitado.

TABLADISPERSAARRAY.C

```
int eliminarSiMenorHashArray(HashArray *hashArray, int minimo);
```

Marca como BORRADA las casillas OCUPADA con elemento menor que minimo. Devuelve cuántas.

Ejercicio 5 · Juntar dos tablas se prueba con `mision5`

TABLADISPERSALISTA.C

```
int fusionarHashLista(HashLista *destino, HashLista *origen);
```

Pasa todo lo de origen a destino moviendo los nodos: si la clave ya está, suma y libera el nodo de origen; si no, engancha ese nodo al principio de su cubo en destino. Origen queda vacío (cubos a NULL y n = 0). Devuelve 1.

TABLADISPERSAARRAY.C

```
int fusionarHashArray(HashArray *destino, HashArray *origen);
```

Recorre las OCUPADA de origen de la 0 a la última: suma si la clave ya está en destino o la inserta. Origen queda vacío (casillas VACIA y n = 0). Devuelve 1, o 0 si alguna no cabe.

Ejercicio 6 · Copiar una tabla se prueba con `mision6`

TABLADISPERSALISTA.C

```
int copiarHashLista(HashLista *destino, HashLista *origen);
```

Libera primero todo lo que tuviera destino y copia cada cubo de origen en el mismo cubo, en el mismo orden, con nodos nuevos, y copia también su función hash. Destino no comparte ningún nodo con origen. Devuelve cuántos elementos ha copiado.

TABLADISPERSAARRAY.C

```
int copiarHashArray(HashArray *destino, HashArray *origen);
```

Copia casilla a casilla las claves, los elementos y las marcas (BORRADA incluidas), y también n, la función hash y el sondeo. Devuelve cuántas OCUPADA ha copiado.

Ejercicio 7 · Comparar dos tablas se prueba con `mision7`

> TABLADISPERSALISTA.C
>
> ```
> int igualesHashLista(HashLista *a, HashLista *b);
> ```
>
> Devuelve 1 si las dos tienen exactamente las mismas claves con los mismos elementos, aunque dentro de un cubo vayan en otro orden; 0 si no. No cambia nada.

> TABLADISPERSAARRAY.C
>
> ```
> int igualesHashArray(HashArray *a, HashArray *b);
> ```
>
> Lo mismo para el array, aunque las claves estén en otras casillas.

Ejercicio 8 · Intersección se prueba con `mision8`

> TABLADISPERSALISTA.C
>
> ```
> int interseccionHashLista(HashLista *destino, HashLista *otro);
> ```
>
> Destino se queda solo con las claves que también están en otro, cada una con el menor de los dos elementos. Los nodos que sobran se liberan y otro no cambia. Devuelve cuántos quedan.

> TABLADISPERSAARRAY.C
>
> ```
> int interseccionHashArray(HashArray *destino, HashArray *otro);
> ```
>
> Lo mismo; lo que se quita pasa a BORRADA.

Ejercicio 9 · Diferencia (restar) se prueba con `mision9`

> TABLADISPERSALISTA.C
>
> ```
> int diferenciaHashLista(HashLista *destino, HashLista *otro);
> ```
>
> Por cada clave de otro que esté en destino, resta su elemento al de destino; si queda en 0 o menos, ese nodo se quita y se libera. Las claves de otro que no están en destino no hacen nada, y otro no cambia. Devuelve cuántos nodos se han quitado.

> TABLADISPERSAARRAY.C
>
> ```
> int diferenciaHashArray(HashArray *destino, HashArray *otro);
> ```
>
> Lo mismo; lo que se queda a 0 o menos pasa a BORRADA.

Ejercicio 10 · Hash de plegamiento se prueba con `mision10`

> TABLADISPERSALISTA.C
>
> ```
> int hashPlegamientoHashLista(int clave);
> ```
>
> Parte la clave en trozos de dos cifras empezando por la derecha, los suma y devuelve la suma mod TAM_LISTA (101). Por ejemplo, 12345 → 1 + 23 + 45 = 69.

> TABLADISPERSAARRAY.C
>
> ```
> int hashPlegamientoHashArray(int clave);
> ```
>
> Lo mismo, mod TAM (20).

Ejercicio 11 · Hash de la mitad del cuadrado se prueba con `mision11`

> TABLADISPERSALISTA.C
>
> ```
> int hashMitadCuadradoHashLista(int clave);
> ```
>
> Eleva la clave al cuadrado en un long long y se queda con las dos cifras del centro: (c / 100) % 100. Devuelve eso mod 101. Por ejemplo, 1234 → 1522756 → 27.

> TABLADISPERSAARRAY.C
>
> ```
> int hashMitadCuadradoHashArray(int clave);
> ```
>
> Lo mismo, mod 20.

Ejercicio 12 · Hash polinómico sobre las cifras se prueba con `mision12`

> TABLADISPERSALISTA.C
>
> ```
> int hashPolinomicoHashLista(int clave);
> ```
>
> h = 0 y, cifra a cifra empezando por la última (clave % 10, luego clave / 10…), h = h * 31 + cifra, con h unsigned int. Devuelve h mod 101.

> TABLADISPERSAARRAY.C
>
> ```
> int hashPolinomicoHashArray(int clave);
> ```
>
> Lo mismo, mod 20.

Ejercicio 13 · Exploración cuadrática y doble hashing se prueba con `mision13`

> TABLADISPERSAARRAY.C
>
> ```
> int hash2HashArray(int clave);
> int casillaHashArray(HashArray *hashArray, int clave, int intento);
> ```
>
> hash2 devuelve 7 - (clave mod 7). casillaHashArray devuelve la casilla del intento número intento según el sondeo de la tabla: lineal (h + i) (ya hecha), cuadrática (h + i·i) o doble (h + i·hash2(clave)), todo mod 20, siendo h la casilla que da la función hash de la tabla. Insertar, buscar, eliminar y actualizar ya la usan.

Ejercicio 14 · Lo que cuesta buscar se prueba con `mision14`

> TABLADISPERSALISTA.C
>
> ```
> int costeBuscarHashLista(HashLista *h, int clave);
> int costeTotalHashLista(HashLista *h);
> ```
>
> costeBuscar: cuántos nodos de su cubo se miran hasta el de la clave (incluido), o todos si no está. costeTotal: la suma del coste de cada clave que hay.

> TABLADISPERSAARRAY.C
>
> ```
> int costeBuscarHashArray(HashArray *h, int clave);
> int costeTotalHashArray(HashArray *h);
> ```
>
> costeBuscar: cuántas casillas se miran siguiendo casillaHashArray desde el intento 0 hasta la de la clave o la primera VACIA, las dos incluidas (como mucho 20). costeTotal: la suma para cada clave que hay.

Ejercicio 15 · Reorganizar se prueba con `mision15`

> TABLADISPERSALISTA.C
>
> `int alFrenteHashLista(HashLista *hashLista, int clave);`
>
> Si la clave está, mueve SU nodo al principio de su cubo, sin malloc ni free. Devuelve 1, o 0 si no está.

> TABLADISPERSAARRAY.C
>
> `int limpiarBorradasHashArray(HashArray *hashArray);`
>
> Guarda aparte las OCUPADA (de la 0 a la última), deja todas las casillas VACIA y n = 0, y las vuelve a insertar en ese orden, sin cambiar el tamaño. Devuelve cuántas BORRADA había.

Rúbrica

La práctica se califica sobre 10. La nota de la web es orientativa; la calificación la pone el profesor revisando el código entregado. Su peso en la calificación final es el que fija la guía docente.

Ejercicios (7 puntos)

Ejercicio	Puntos	Ejercicio	Puntos
Ejercicio 1 · El elemento más grande	**0,25**	Ejercicio 9 · Diferencia (restar)	**0,5**
Ejercicio 2 · Medir las colisiones	**0,5**	Ejercicio 10 · Hash de plegamiento	**0,25**
Ejercicio 3 · Cambiar la clave	**0,5**	Ejercicio 11 · Hash de la mitad del cuadrado	**0,25**
Ejercicio 4 · Quitar los que tienen poco	**0,5**	Ejercicio 12 · Hash polinómico sobre las cifras	**0,5**
Ejercicio 5 · Juntar dos tablas	**0,75**	Ejercicio 13 · Exploración cuadrática y doble hashing	**0,5**
Ejercicio 6 · Copiar una tabla	**0,5**	Ejercicio 14 · Lo que cuesta buscar	**0,5**
Ejercicio 7 · Comparar dos tablas	**0,25**	Ejercicio 15 · Reorganizar	**0,75**
Ejercicio 8 · Intersección	**0,5**		
Total de los ejercicios			**7**

Cada ejercicio se puntúa con uno de estos tres niveles:

Nivel	Puntos	Cuándo
Completo	Todos	Hace lo que pide su descripción en todas sus tablas y en todos los casos, sin fugas ni otros errores de memoria.
Parcial	La mitad	Funciona en parte: falla algún caso, solo funciona en una de las dos tablas o tiene algún error de memoria.
No alcanzado	0	No está, no compila o no hace lo que se pide.

Calidad del código (3 puntos)

Criterio	Puntos	Completo	Parcial (la mitad)	No alcanzado (0)
Claridad	**1**	Nombres que se entienden, código ordenado y comentarios donde ayudan.	Se entiende con esfuerzo.	Difícil de seguir.
Sin repeticiones ni sobrantes	**1**	Lo que se repite va en una función auxiliar; no hay variables ni casos de sobra.	Alguna repetición o resto innecesario.	Mucho código copiado o sobrante.
Compila limpio	**1**	Sin avisos con `-Wall -Wextra -pedantic`.	Algún aviso.	Muchos avisos.
Total de la práctica	**10**	**7 de los ejercicios + 3 de calidad del código**		